

Lecture: 0110

Registers, Counters & State Machines

Building Memory That Does Something

Chain flip-flops together and a circuit can store words, count, and follow a plan.

CSA222: Modern Computer Architecture

One Flip-Flop = One Bit

L5 gave us a reliable 1-bit memory. Today we build things worth remembering.



The D flip-flop

Captures one bit on the clock edge, holds it for the cycle (L5).



Edge-triggered & safe

Master-slave design blocks data races — predictable updates.



Timing rules

Setup, hold, metastability — the constraints that keep it working (L5).



But just ONE bit...

A CPU needs 32-bit words, counters, and control. So we chain them.

The jump from 'store a bit' to 'store a word, count events, run a program' is what today is about.

Today's Roadmap

Three structures, each more capable than the last.

1

Registers

n flip-flops in parallel = an n-bit word store. Add shifting for data movement.



2

Counters

Flip-flops that count clock pulses — the basis of timers and addresses.



3

State Machines

The general model of any sequential circuit: states, transitions, outputs.

The big idea: an FSM is the universal blueprint — registers and counters are just special cases of it. It's how a CPU 'thinks'.



when studying
computer engineering
goes from this...





The Flip-Flop Toolkit

Four flip-flops, four truth tables — and the one reverse-lookup table that makes design possible.

PRIMARY DEFINITION

A flip-flop is a one-bit memory that updates only on a clock edge. Change what feeds its input and you change what it does.

SR Set / Reset — the original latch

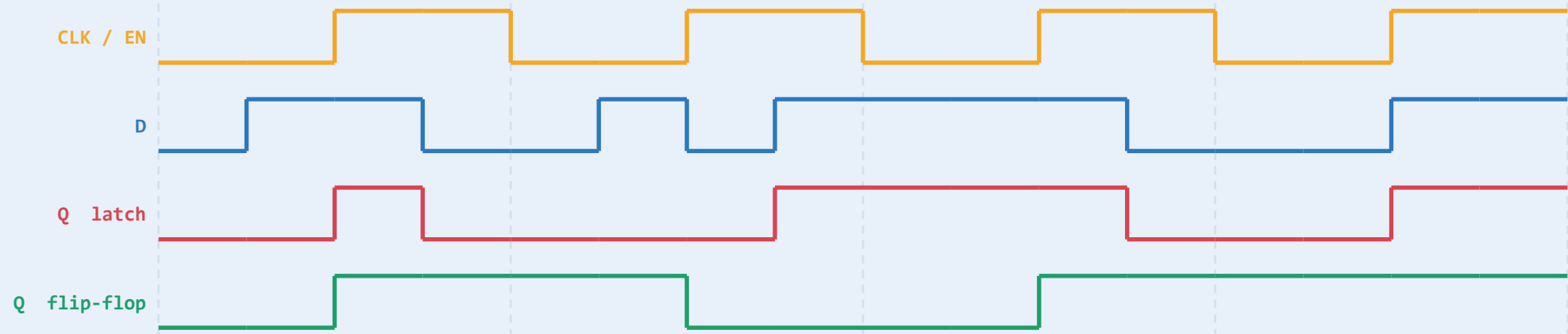
D Delay — copies its input, one cycle later

JK The universal one — SR with toggle

T Toggle — flips on command. Counters live here.

Latch vs Flip-Flop

Both remember a bit. The difference is WHEN they are allowed to change.



Same clock, same data — two very different outputs. The latch copies D the whole time EN is high; the flip-flop looks once, at the edge.

LEVEL-TRIGGERED — LATCH

Transparent: while the enable is high, Q follows D continuously. Any wobble on D passes straight through.

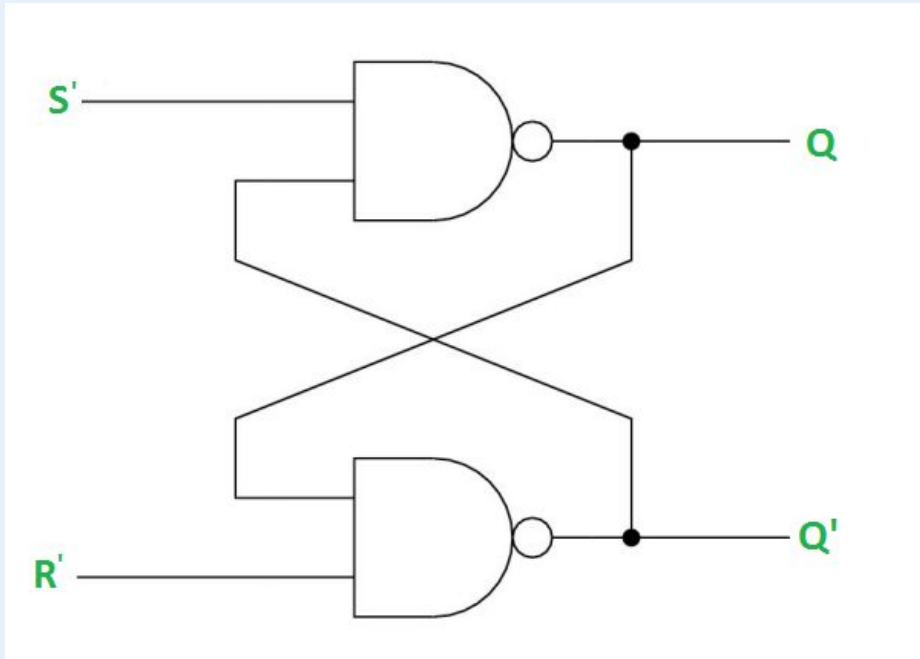
EDGE-TRIGGERED — FLIP-FLOP

Q samples D at one instant only, then holds for the whole cycle. This is why we can reason about timing at all.

Everything from here on uses edge-triggered flip-flops. When this lecture says 'flip-flop', assume the rising clock edge.

The SR Latch — Where Memory Starts

Two NOR gates feeding each other. The simplest circuit that can remember.



CHARACTERISTIC TABLE

S	R	Q^+	MEANING
0	0	Q	Hold — remembers the last value
0	1	0	Reset the bit to 0
1	0	1	Set the bit to 1
1	1	—	Forbidden — both outputs go 0

THE FLAW

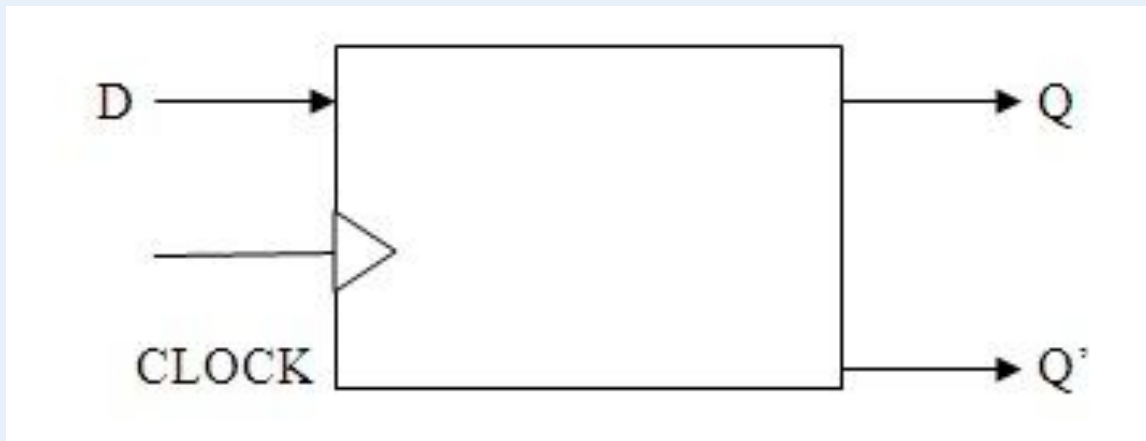
$S = R = 1$ breaks the promise that Q and Q' are opposites — and when both inputs drop together the result is unpredictable.

Fixing that forbidden row is exactly what the JK flip-flop is for — and the fix turns out to give us the toggle we need for counters.

The D Flip-Flop — One Input, No Surprises

Tie S and R together through an inverter and the forbidden state becomes unreachable.

SYMBOL & PIN NAMES



CHARACTERISTIC TABLE

Q	D	Q(n+1)	WHAT HAPPENED
0	0	0	stayed at 0
0	1	1	loaded a 1
1	0	0	loaded a 0
1	1	1	stayed at 1

CHARACTERISTIC EQUATION

$Q(n+1) = D$ — the next state is simply whatever D was at the edge.

WHY IT IS THE DEFAULT

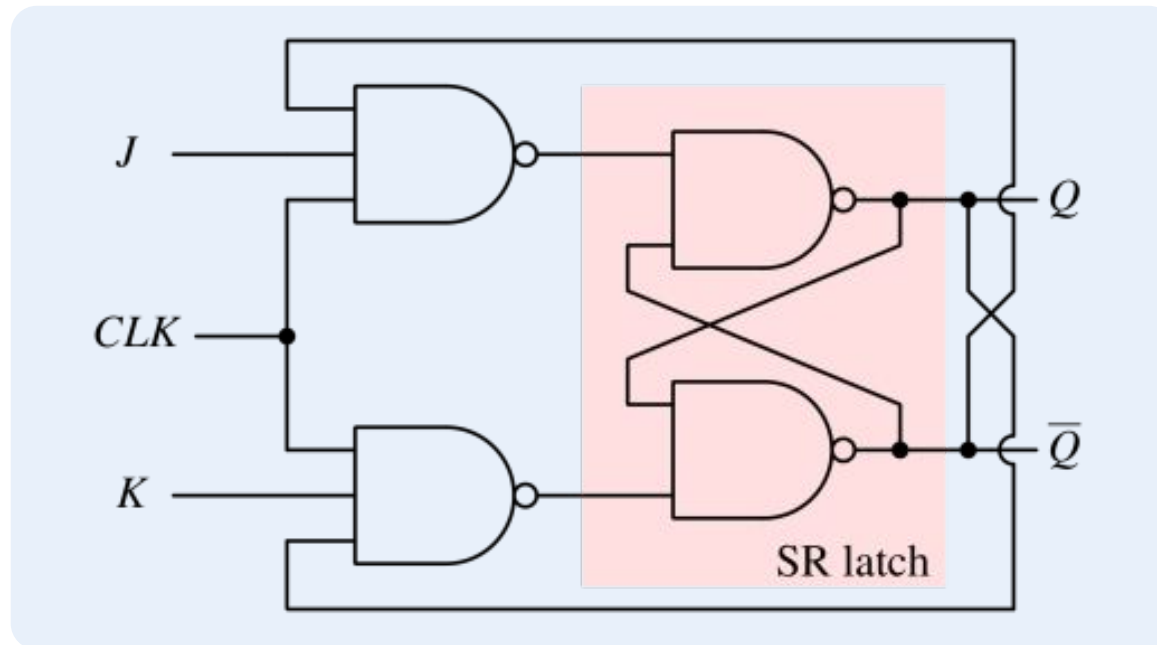
One data input means one thing can go wrong instead of four. Every register, pipeline stage and state register in a modern CPU is built from D flip-flops.

THE COST

A D flip-flop cannot toggle by itself — it only copies. To make it flip, you must feed it the opposite of its own output. Hold that thought.

The JK Latch — Reclaiming the Fourth Row

Same four input combinations as SR, but the illegal one is redefined as 'toggle'.



CHARACTERISTIC TABLE

J	K	Q^+	MEANING
0	0	Q	Hold — no change
0	1	0	Reset to 0
1	0	1	Set to 1
1	1	Q'	TOGGLE — flip to the opposite

CHARACTERISTIC EQUATION

$$Q(n+1) = J\bar{Q}(n) + Q\bar{K}(n)$$

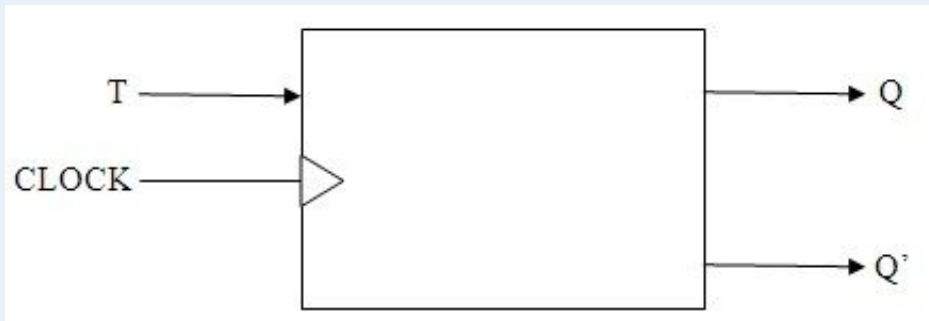
WHY THIS MATTERS TODAY

The JK flip-flop can do everything SR and D can do, plus one new trick: toggle. Toggling is counting. Tie J and K together and you get the flip-flop the rest of this lecture depends on.

The T Flip-Flop — The One Counters Are Made Of

A single input with a single job: $T = 1$ means 'flip', $T = 0$ means 'stay'.

SYMBOL



CHARACTERISTIC TABLE

T	Q	Q ⁺	ACTION
0	0	0	hold
0	1	1	hold
1	0	1	flip
1	1	0	flip

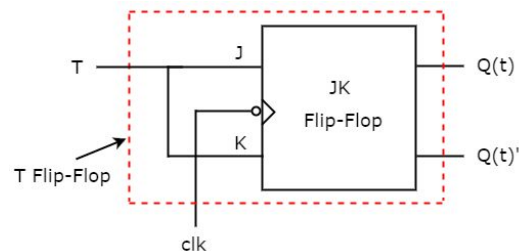
Remember this device. Every counter in the next two sections is a row of T flip-flops with different logic deciding when each one flips.

CHARACTERISTIC EQUATION

$$Q(n+1) = TQn' + T'Qn = T \text{ XOR } Qn$$

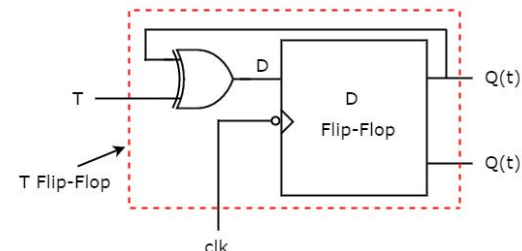
TWO WAYS TO BUILD ONE

1 • From a JK flip-flop



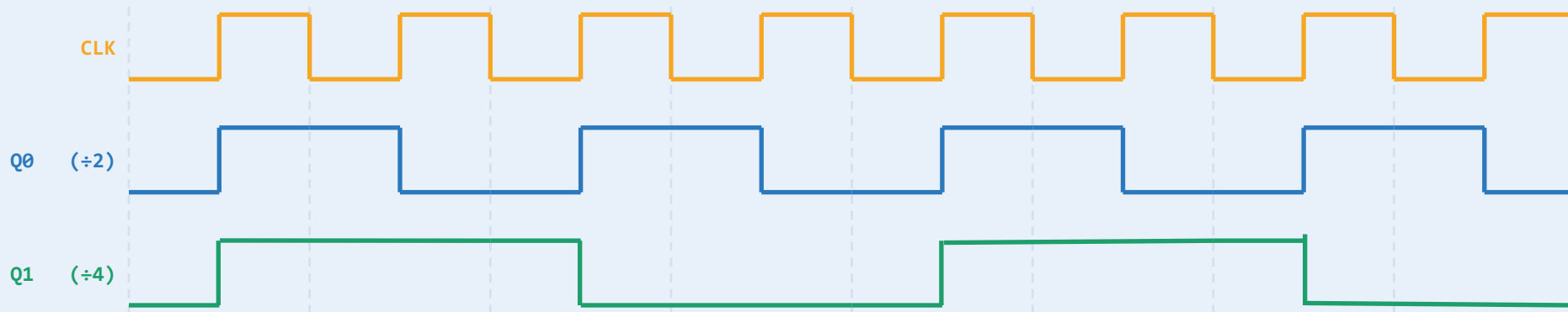
Tie J and K to the same wire.

2 • From a D flip-flop



Hold T at 1 and You Get a Divide-by-2

A permanently toggling flip-flop halves the clock frequency. Chain them and you have a counter.



Q0 completes one cycle for every two clock cycles. Q1 is clocked by Q0, so it completes one cycle for every four. Each extra flip-flop halves the frequency again — and the bits you are looking at spell out 0, 1, 2, 3 in binary.

AS A DIVIDER

n stages divide the clock by 2^n . This is how a 32 768 Hz watch crystal becomes a 1 Hz tick.

AS A COUNTER

Read the stages as bits and the same circuit is counting up in binary — same wires, different reading.

THE LINK

Counting and dividing are the same operation. That is why one circuit serves timers, PWM and program counters.

The Four Flip-Flops at a Glance

One table to keep. Everything else in this lecture is built from these four rows.

TYPE	INPUTS	WHAT IT DOES	CHARACTERISTIC EQ.	TYPICALLY USED FOR
SR	S, R	Set, reset or hold — but $S = R = 1$ is illegal	$Q^* = S + R' \cdot Q$	Latches, simple control
D	D	Copies its input to the output on the edge	$Q^* = D$	Registers, pipeline stages, state registers
JK	J, K	Set, reset, hold — and toggle when both are 1	$Q^* = J \cdot Q' + K' \cdot Q$	General-purpose sequential design
T	T	Flips when $T = 1$, holds when $T = 0$	$Q^* = T \text{ XOR } Q$	Counters and frequency dividers

THEY ARE ALL THE SAME DEVICE

Every one of these is an SR latch pair with different logic bolted onto the front. Choosing a flip-flop type is really choosing how much of the next-state logic you want built in already.

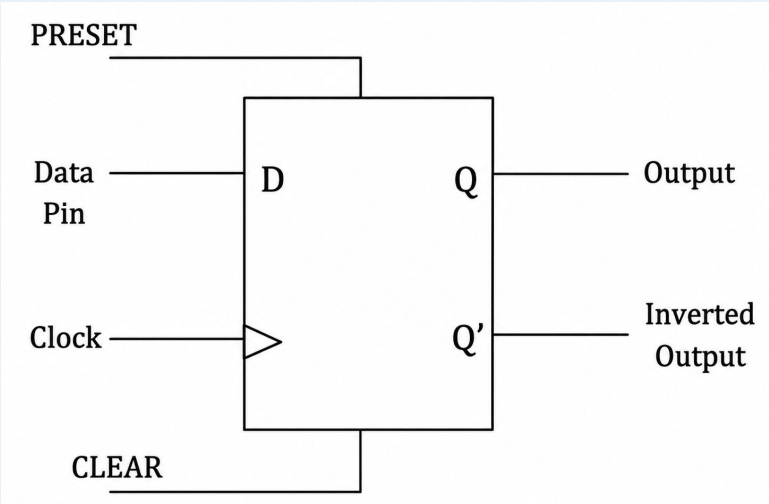
WHAT REAL HARDWARE USES

Modern chips and every FPGA use D flip-flops almost exclusively — they are the smallest and fastest. JK and T are design conveniences: you reason in T, the synthesis tool builds it out of D and an XOR.

Preset, Clear and Reset

Every real flip-flop has a back door that ignores the clock. Counters and state machines both need it.

THE EXTRA PINS



	ASYNCHRONOUS	SYNCHRONOUS
When it acts	Immediately	On the next clock edge
Needs the clock?	No	Yes
Timing risk	Can violate setup/hold	None — it is ordinary logic
Typical use	Power-on reset	Mod-N counter wrap, FSM restart

Rule of thumb: one asynchronous reset for power-on, synchronous resets for everything the design does while it is running.

WITHOUT A RESET

Flip-flops power up in a random state. A counter might start at 173, and a state machine might start in a state that has no way out. The circuit works perfectly in simulation and hangs on the bench.

WITH A RESET

One known starting point, guaranteed. Every FSM you draw from now on gets a reset arrow pointing at its initial state — no exceptions.

1

Registers

The foundational building blocks of CPU storage.

PRIMARY DEFINITION

A register is just a row of flip-flops sharing a clock — the **fastest, closest memory** a CPU has.



Flip-Flop Row
Multi-bit storage



Shared Clock
Atomic updates



Fastest Speed
Closest to CPU

From One Bit to Many

Put flip-flops side by side, share a clock, and you store a whole word at once.

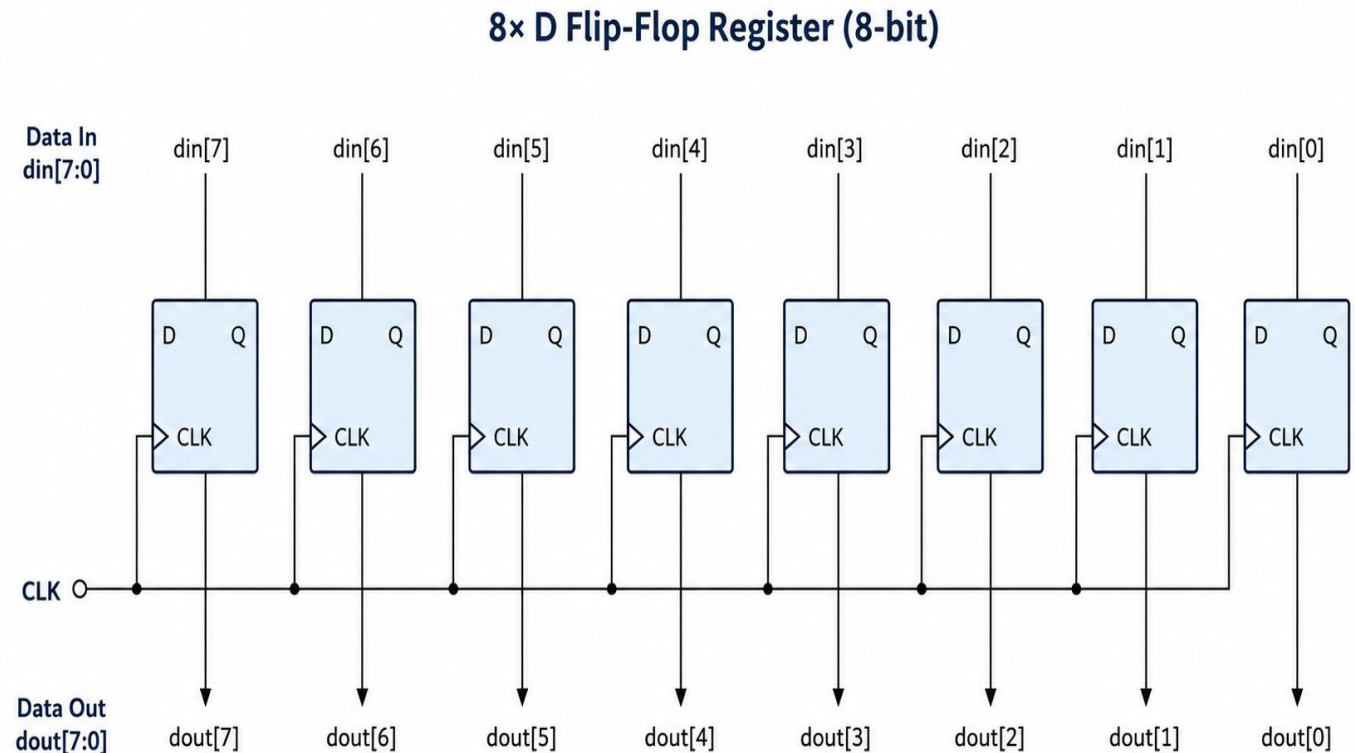
Atomic Multi-Bit Storage

*All four bits are captured on the **SAME clock edge** — the word updates atomically, as one single unit.*

THE SYSTEM SECRET

What is a CPU register?

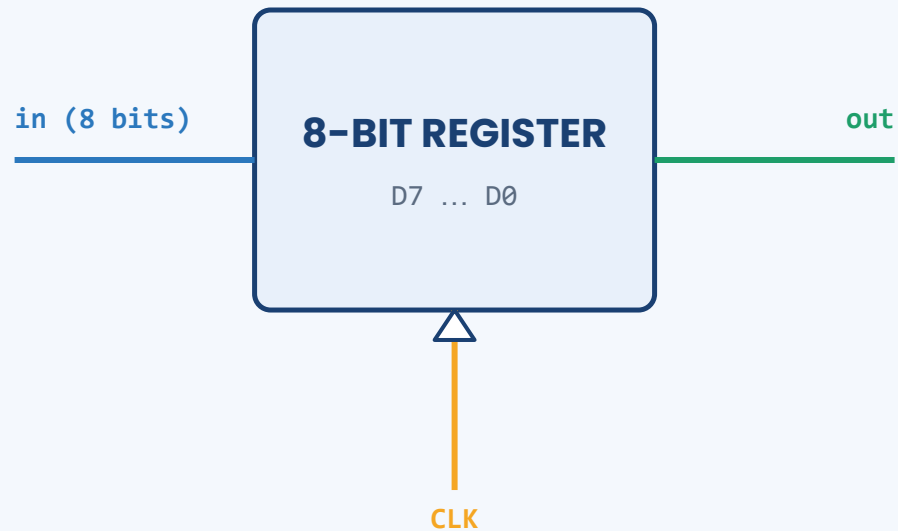
This is literally what a CPU register is: **32 or 64 D-flip-flops** sharing a single, synchronous clock. That is the entire secret.



The Register

Store, hold, and read back an n-bit value — the CPU's working scratchpad.

BLOCK DIAGRAM



WHERE REGISTERS LIVE

The register file — 32 registers in MIPS/RISC-V — is a CPU's fastest storage.

It serves as the immediate staging area for active operations, feeding data directly to execution units.

Also: the program counter (PC), instruction register, and pipeline stage registers are all just registers.

Faster than cache, faster than RAM — because it's flip-flops right next to the ALU.

The Load Enable

A register must HOLD its value across clock edges until you choose to update it.

THE PROBLEM

A plain flip-flop grabs D on EVERY clock edge. But a register should keep its value for many cycles, updating only when told.

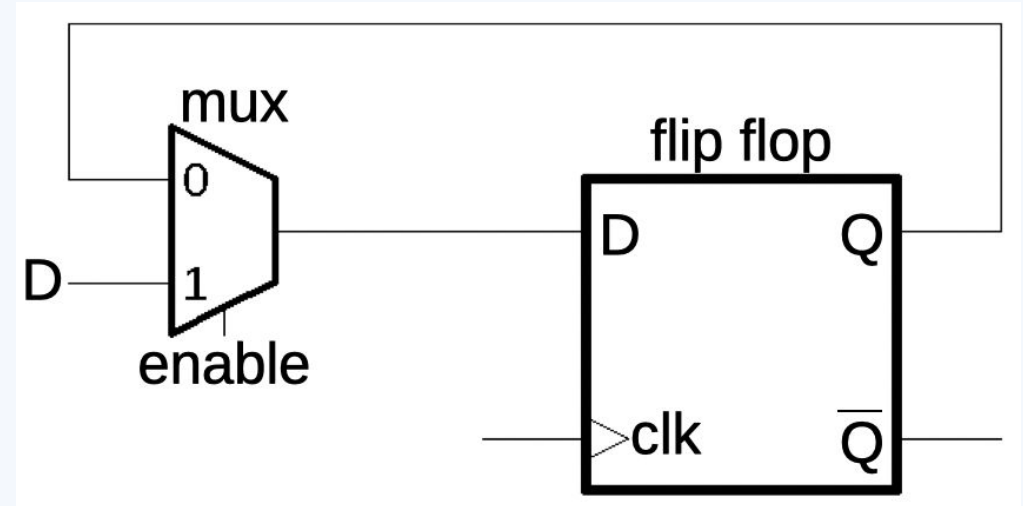
THE FIX — a load-enable MUX per bit:

LOAD = 1 → feed new data in.

LOAD = 0 → feed the flip-flop's OWN output back in (hold).

A 2:1 MUX on each bit's D input chooses between 'new value' and 'keep current'.

ONE BIT'S PATH

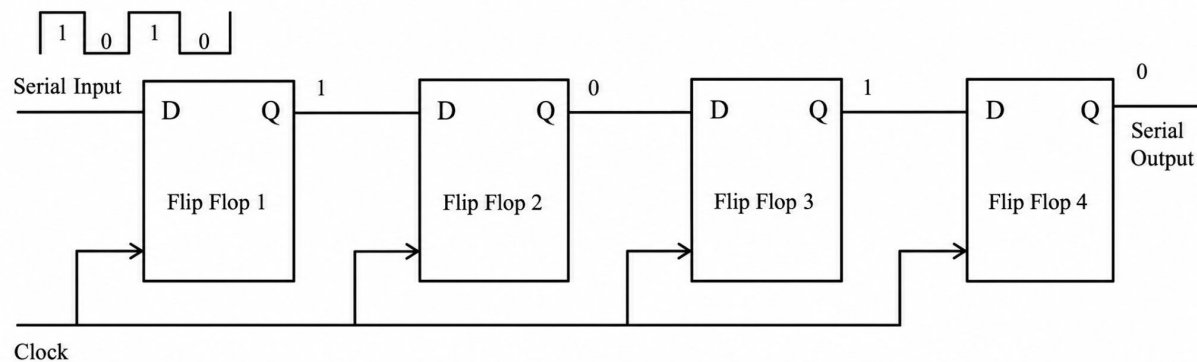


The MUX + feedback loop = a register that remembers until LOAD says otherwise.

The Shift Register

Chain flip-flops output-to-input: each clock edge, every bit shifts one place.

BLOCK DIAGRAM



HOW IT SHIFTS

Each flip-flop's output feeds the NEXT one's input. A bit entered at 'in' walks right, one FF per clock.

It's a conveyor belt for bits — the basis of serial data transfer and multiply/divide-by-2 operations.

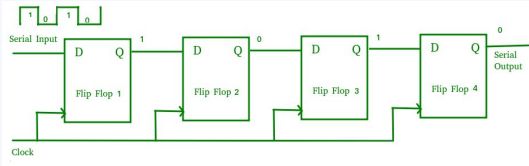
Four Shift Register Types

Named by how data goes IN and comes OUT: serial or parallel.

SISO

Serial In, Serial Out

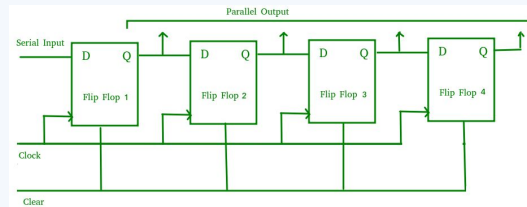
One bit in, one bit out per clock — a pure delay line.



SIPO

Serial In, Parallel Out

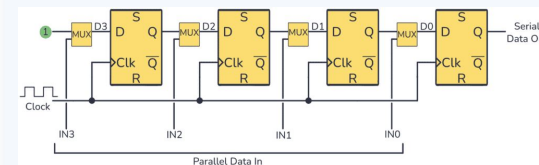
Bits enter one at a time, read out all at once — serial→parallel converter.



PISO

Parallel In, Serial Out

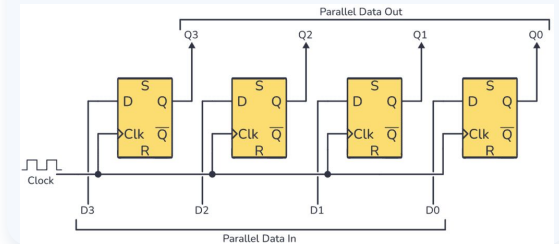
Load a whole word, shift it out bit by bit — parallel→serial converter.



PIPO

Parallel In, Parallel Out

Load and read a full word — this is just a normal register with load.



SIPO and PISO are the workhorses: they convert between serial links (few wires) and parallel buses (fast).

What Shift Registers Do

Small circuit, surprisingly many jobs.



Serial ↔ parallel

UART, SPI, USB all send data one bit at a time over few wires; shift registers reassemble it into bytes.



Multiply / divide by 2

Shifting binary left multiplies by 2; shifting right divides by 2. Free arithmetic, no adder.



Delay lines & buffers

A SISO register delays a signal by exactly N clock cycles — used in DSP and pipelines.



Pseudo-random (LFSR)

Add feedback via XOR and a shift register generates long pseudo-random sequences — used in crypto & test.



**Thanks
for
watching!**